MA 587: Homework 1

Joseph F. Specht IV

Febuary 13, 2026

1 Mollifier

1.1 Continuousness

1. For this problem, we need to show η is continuous at ± 1. However, we see $\eta(-x) = \eta(x)$ and the function η is symmetric about the origin. Therefore, we only need to show η is continuous around either $x = 1$ or $x = -1$ to show η is continuous at ± 1.

2. For a function to be continuous at a given x, the function needs to be defined at x and have the limit from both sides equal the value at x. We see $\eta(\pm 1) = 0$, so we need show the limit from both sides equals 0.

3. Limit from the right:

$$\lim_{x \to 1^+} = (\eta(x), \quad \text{if } |x| \geq 1) \tag{1a}$$

$$\lim_{x \to 1^+} = 0 \tag{1b}$$

4. Limit from the left:

$$\lim_{x \to 1^-} = (\eta(x), \quad \text{if } |x| < 1) \tag{2a}$$

$$\lim_{x \to 1^-} = \exp\left(-\frac{1}{1-x^2}\right) = 0 \tag{2b}$$

5. We conclude η is continuous as:

$$\eta(x) = \lim_{x \to 1^+} = \lim_{x \to 1^-} = 0 \tag{3}$$

1.2 Differentiability

1. We need to show η is infinitely differentiable. We can do so by checking if η is differentiable from both sides. Similarly to the continuity, we know η is an even function, so if $\eta(x)$ is infinitely differentiable,

so is $\eta(-x)$. We will only consider $\eta(1)$.

2. From the right, we see the 0 is infinitely differentiable.

3. From the left, we can take the derivative:

$$\eta'(x) = -\frac{2x}{(1-x^2)^2}\exp\left(-\frac{1}{1-x^2}\right) = \frac{P_1}{P_4}\exp\left(-\frac{1}{1-x^2}\right) = \frac{1}{P_3}\exp\left(-\frac{1}{1-x^2}\right) \tag{4}$$

   (a) where P_i is a polynomial of order i.

4. Taking the second derivative :

$$\eta(x)'' = \frac{P_4}{P_8}\exp\left(-\frac{1}{1-x^2}\right) = \frac{1}{P_4}\exp\left(-\frac{1}{1-x^2}\right) \tag{5}$$

5. The third derivative:

$$\eta(x)''' = \frac{P_7}{P_{12}}\exp\left(-\frac{1}{1-x^2}\right) = \frac{1}{P_5}\exp\left(-\frac{1}{1-x^2}\right) \tag{6}$$

6. We see the general pattern for derivatives of η:

$$\eta^{(i)} = \frac{\exp\left(-\frac{1}{1-x^2}\right)}{P_{i+2}} \tag{7}$$

7. To check for continuity, we can use L'Hospital's rule, which states the limit of a ratio of two functions is equal to the ratio of the derivatives of each function. For this example, we have an exponential function in the numerator and some polynomial of arbitrary order in the bottom:

$$\lim_{x\to 1^+}\eta^{(i)} = \lim_{x\to 1^+}\frac{\exp\left(-\frac{1}{1-x^2}\right)}{P_{i+2}} = \lim_{x\to 1^+}\frac{d^n}{dx^n}\left[\frac{\exp\left(-\frac{1}{1-x^2}\right)}{P_{i+2}}\right] \tag{8}$$

8. From this form, it is evident the polynomial denominator can be infinitely differentiated away leaving only the exponential function in the numerator. Because you can infinitely differentiate this exponential function, we conclude η is infinitely differentiable at $x = \pm 1$

2 Fundamental Lemma of the Calculus of Variations – FLCV

1. Because $v \in V$, we know some v can be non-zero at any given point on $(0,1)$

2. Let's do a proof by contradiction and assume at least one point $c \in (0,1)$ gives $w(c) \neq 0$

3. Next, choose $\epsilon = \frac{w(c)}{2}$, so
$$|w(y) - w(c)| < \frac{w(c)}{2} \quad \forall y \in (c-\delta, c+\delta) \tag{9}$$

4. We select a the mollifier with the form of a parabola that has x-intercepts at $c \pm \delta$.
$$v(x) = \begin{cases} [x-(c-\delta)]\,[(c+\delta)-x]\,, & \in (c-\delta, c+\delta) \\ 0, & \text{else} \end{cases} \tag{10}$$

5. We can assume $w(c) > 0$ to set up the proof by contradiction. We also know $v(x) = 0$ outside of $(c-\delta, c+\delta)$, so the integral in this region is 0. However, the entire integral is still equal to zero.
$$\int_0^1 dx w(x)v(x) = \int_{c-\delta}^{c+\delta} dx w(x)v(x) = 0 \tag{11}$$

6. From the definition, $v \neq 0$ over the bounder of integration. From this, we notice our contradiction in assuming there is some $w(c) \neq 0$. Because we can set $v(x)$ to be non-zero at any location $\in (0,1)$ using the same form as we used in the mollifier expression, we see $w(c) \neq 0$ is a wrong assumption.

7. Because $w(x)$ needs to apply for all $v(x)$, we can imagine if $w(x)$ is non-zero anywhere, we can devise a $v(x)$ such that $\int wv \neq 0$.

8. $\therefore$ if $w(x)$ is continuous over $(0,1)$ and $\int_0^1 dx w(x)v(x) = 0 \quad \forall v \in V$, $w(x) = 0$ because we can set $v(x)$ to non-zero at any point over $(0,1)$, so $w(x)$ needs to be zero to maintain the imposition of $\int wv = 0$.

3 Boundary Value Problem

3.1 Variational Form

1. We know $u^{(4)} = f$, so $\int_0^1 dx u^{(4)} v = \int_0^1 dx f v$

2. We apply integration by parts twice to obtain a new form of the left-hand side:

$$\int_0^1 dx u^{(4)} v = [u''' v]_0^1 - \int_0^1 dx u''' v' = [u''' v]_0^1 - \left([u'' v']_0^1 - \int_0^1 dx u'' v'' dx \right) \tag{12}$$

3. The first and second terms cancel when we apply the boundary conditions $v(0) = v(1) = 0$ and $v'(0) = v'(1) = 0$, respectively.

4. This leaves us with the expected form:

$$\int_0^1 dx u'' v'' = \langle u'', v'' \rangle = \langle f, v \rangle \tag{13}$$

3.2 Coefficients

We know each basis function is a cubic polynomial, which means the derivative of each basis function is a quadratic polynomial.

3.2.1 ϕ_L

1. We start with a general form for ϕ_L'

$$\phi_L'(x) = A_1 (x - a)(x - b) \tag{14}$$

2. Integrating to obtain ϕ_L

$$\phi_L = A_1 \left[\frac{x^3}{3} - \frac{x^2}{2}(a + b) + xab \right] + C_1 \tag{15}$$

3. Plugging in the boundary conditions:

$$\phi_L(a) = 1 = A_1 \left[\frac{a^3}{3} - \frac{a^2}{2}(a + b) + a^2 b \right] + C_1 \tag{16a}$$

$$\phi_L(b) = 0 = A_1 \left[\frac{b^3}{3} - \frac{b^2}{2}(a + b) + ab^2 \right] + C_1 \tag{16b}$$

4. We can subtract the two previous equations to cancel C_1 obtain an expression for A_1

5. Once we have an expression for A_1, we can solve for C_1 in terms of A_1.

We find ϕ_L is given as:

$$\phi_L(x) = A_1 \left[\frac{x^3}{3} - \frac{x^2}{2}(a+b) + xab \right] + C_1 \tag{17a}$$

$$A_1 = \left[\frac{(a^3 - b^3)}{3} - \frac{(a+b)}{2}(a^2 - b^2) + ab(a-b) \right]^{-1} \tag{17b}$$

$$C_1 = -A \left[\frac{b^3}{3} - \frac{b^2}{2}(a+b) + ab^2 \right] \tag{17c}$$

3.2.2 ϕ_R

1. Similar to ϕ_L, we start with a general form for ϕ'_R (same form as ϕ'_L)

$$\phi'_R(x) = A_2(x-a)(x-b) \tag{18}$$

2. Integrating to obtain ϕ_R

$$\phi_R = A_2 \left[\frac{x^3}{3} - \frac{x^2}{2}(a+b) + xab \right] + C_2 \tag{19}$$

3. Plugging in the boundary conditions:

$$\phi_R(a) = 0 = A_1 \left[\frac{a^3}{3} - \frac{a^2}{2}(a+b) + a^2 b \right] + C_2 \tag{20a}$$

$$\phi_R(b) = 1 = A_1 \left[\frac{b^3}{3} - \frac{b^2}{2}(a+b) + ab^2 \right] + C_2 \tag{20b}$$

4. We can subtract the two previous equations to cancel C_2 obtain an expression for A_2

5. Once we have an expression for A_2, we can solve for C_2 in terms of A_2.

We find ϕ_R is given as:

$$\phi_R(x) = A_2 \left[\frac{x^3}{3} - \frac{x^2}{2}(a+b) + xab \right] + C_2 \tag{21a}$$

$$A_2 = \left[\frac{(b^3 - a^3)}{3} - \frac{(a+b)}{2}(b^2 - a^2) + ab(b-a) \right]^{-1} \tag{21b}$$

$$C_2 = -A \left[\frac{a^3}{3} - \frac{a^2}{2}(a+b) + a^2 b \right] \tag{21c}$$

3.2.3 ψ_L

1. We start with a general form of ψ_L, which we know is a quadratic function, so one of the quantities $(x-a)$ or $(x-b)$ needs to be squared. If we chose wrong and raised $(x-b)^2$ instead, we would have written the expression for ψ_R:

$$\psi_L = A_3(x-a)^2(x-b) \tag{22}$$

2. Differentiate and substitute the boundary condition

$$\psi_L' = A_3(x-a)(3x-2b-a) \tag{23a}$$

$$\psi_L'(a) = 1 = A_3(a-b)(a-b) \tag{23b}$$

We can solve for A_3 and find ψ_L is given as:

$$\psi_L(x) = A_3(x-b)^2(x-a) \tag{24a}$$

$$A_3 = (a-b)^{-2} \tag{24b}$$

3.2.4 ψ_R

1. We start with a general form of ψ_R:

$$\psi_R = A_4(x-a)(x-b)^2 \tag{25}$$

2. Differentiate and substitute the boundary condition

$$\psi_R' = A_4(x-b)(3x-2a-b) \tag{26a}$$

$$\psi_R(b)' = 1 = A_4(a-b)(a-b) \tag{26b}$$

We can solve for A_4 and find ψ_R is given as:

$$\psi_R(x) = A_4(x-a)^2(x-b) \tag{27a}$$

$$A_4 = A_3 = (a-b)^{-2} \tag{27b}$$

3.2.5 Decompose some v

1. We will assume some vector v can be written as the orthogonal decomposition given as:

$$v(x) = a_3x^3 + a_2x^2 + a_1x + a_0 \tag{28}$$

2. Which have basis vectors that span P_3, such that span $\{1, x, x^2, x^3\} = P_3$

3. If the previously defined basis vectors span P_3, we have can write any v as a decomposition of these basis vectors.

$$v(x) = c_1\phi_L + c_2\phi_R + c_3\psi_L + c_4\psi_R \tag{29a}$$

$$v(x)' = c_1\phi'_L + c_2\phi'_R + c_3\psi'_L + c_4\psi'_R \tag{29b}$$

4. Substitute boundary conditions and also use the boundary conditions we were given for the basis vectors:

$$v(a) = c_1(1) + c_2(0) + c_3(0) + c_4(0) = c_1 \tag{30a}$$

$$v(b) = c_1(0) + c_2(1) + c_3(0) + c_4(0) = c_2 \tag{30b}$$

$$v'(a) = c_1(0) + c_2(0) + c_3(1) + c_4(0) = c_3 \tag{30c}$$

$$v'(b) = c_1(0) + c_2(0) + c_3(0) + c_4(1) = c_4 \tag{30d}$$

5. Because the different boundary conditions evaluated to different values, we see the previously defined basis vectors are linearly independent. As there are four of these basis vectors, we know they span P_3 because there are a maximum of four degrees of freedom.

3.3 Basis Functions

We choose $W_h \in W$ to be a subspace with the following properties:

$$W_h = \left\{ v : \begin{array}{l} v \text{ is continuous on } (0,1) \\ v \text{ is a piecewise cubic polynomial in each cell} \\ v(0) = v(1) = v'(0) = v'(1) = 0 \end{array} \right\} \tag{31}$$

We previous showed you can write any v as the decomposition of the basis functions we found and $v_h(x)$ is

no exception:

$$v_h(x) = \sum_{j=1}^{M} c_1 \phi_L^{I_j} + c_2 \phi_L^{I_r} + c_3 \psi_L^{I_j} + c_3 \psi_R^{I_j} \tag{32}$$

3.4 FEM Method

We can formulate a finite element method based on W_h to get the linear system of equations. It should be noted these equations are only for cell I_j, but should be defined for every cell.

$$\langle \phi_L^{I_j}, u \rangle = \langle f, \phi_L^{I_j} \rangle \tag{33a}$$

$$\langle \phi_R^{I_j}, u \rangle = \langle f, \phi_R^{I_j} \rangle \tag{33b}$$

$$\langle \psi_L^{I_j}, u \rangle = \langle f, \psi_L^{I_j} \rangle \tag{33c}$$

$$\langle \psi_R^{I_j}, u \rangle = \langle f, \psi_R^{I_j} \rangle \tag{33d}$$

We can then write the full FEM matrix as:

$$\begin{bmatrix} (\phi_L^{I_j}{}'', \phi_L^{I_j}{}'') & (\phi_R^{I_j}{}'', \phi_L^{I_j}{}'') & (\psi_L^{I_j}{}'', \phi_L^{I_j}{}'') & (\psi_R^{I_j}{}'', \phi_L^{I_j}{}'') \\ (\phi_L^{I_j}{}'', \phi_R^{I_j}{}'') & (\phi_R^{I_j}{}'', \phi_R^{I_j}{}'') & (\psi_L^{I_j}{}'', \phi_R^{I_j}{}'') & (\psi_R^{I_j}{}'', \phi_R^{I_j}{}'') \\ (\phi_L^{I_j}{}'', \psi_L^{I_j}{}'') & (\phi_R^{I_j}{}'', \psi_L^{I_j}{}'') & (\psi_L^{I_j}{}'', \psi_L^{I_j}{}'') & (\psi_R^{I_j}{}'', \psi_L^{I_j}{}'') \\ (\phi_L^{I_j}{}'', \psi_R^{I_j}{}'') & (\phi_R^{I_j}{}'', \psi_R^{I_j}{}'') & (\psi_L^{I_j}{}'', \psi_R^{I_j}{}'') & (\psi_R^{I_j}{}'', \psi_R^{I_j}{}'') \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ c_3 \\ c_4 \end{bmatrix} = \begin{bmatrix} \phi_L^{I_j}, f \\ \phi_R^{I_j}, f \\ \psi_L^{I_j}, f \\ \psi_R^{I_j}, f \end{bmatrix} \tag{34}$$

4 FEM Implementation

The code will be broken up and given piecemeal in each of the sections, but the entire file is given in the appendix.

Gemini Prompt: Hello robot, I am doing a finite element homework and need to write an implementation of FEM for one of the questions. This problem is a boundary value problem such that is satisfies the following conditions: - <u",v">=<f,v> on (0,1) - u(0)=u'(0)=u(1)=u'(1)=0

Also, assume the following for this problem:

1. Assume the mesh is equidistant (i.e. h_j = h for all j).
2. let u(x) = sin$\hat{2}$(pi x)
3. the load vectors should be a piecewise-continuous, cubic functions
4. use scipy for integration
5. remember np.trapz has been depreciated in favor of np.trapezoid

Write a python script that does the following in a nice, organized manner with proper sectioning:

a) finds f, where d$\hat{4}$u/dx$\hat{4}$ = f

b) write code to construct the basis vectors according to the first screen shot I attached. additionally, define the second second derivatives of each basis vector

c) write code to construct the stiffness matrix (call it A) from the second screenshot

d) write code to approximate the load vector (call it b) from the second screen shot using Simpson's quadrature rule

e) write code to solve A xi = b and returns the FEM solution. Here, u_h(x) = Sum_j=1$\hat{M}$ xi_i* phi_j(x), where xi* is the solution to A xi = b and phi_j's are the basis functions from part b

f) write code to plot the relative L2 error against the analytical solution. Use 10e5 equidistant points for the spatial discretization of (0,1) . Iterate through M=5,10,...,100 with a step size of 5 and record the relative L2 error

4.1 Part a

```
# ========================================
# Part (a): Analytical Functions
# ========================================
def u_exact(x):
    return np.sin(np.pi * x)**2


def f_analytical(x):
    return -8 * (np.pi**4) * np.cos(2 * np.pi * x)
```

4.2 Part b

For this part, I also asked the robot for the second derivatives of the basis functions. I provided a screenshot of my latex equations for the robot to transcribe.

```
# ========================================
# Part (b): Basis Functions from Screenshot 1
# ========================================
class HermiteBasis:
    def __init__(self, a, b):
        self.a, self.b = a, b
        # Constants for phi_L and phi_R
        denom_phi = (a**3 - b**3)/3 - (a+b)/2 * (a**2 - b**2) + a*b*(a-b)
        self.A1 = 1.0 / denom_phi
        self.C1 = -self.A1 * (b**3/3 - (a+b)/2 * b**2 + a*b**2)

        self.A2 = -self.A1 # From symmetry/screenshot 2b
        self.C2 = -self.A2 * (a**3/3 - (a+b)/2 * a**2 + a**2*b)

        # Constants for psi_L and psi_R
        self.A3 = (a - b)**(-2)
        self.A4 = self.A3

    def phi_L(self, x):
```

```
        return self.A1 * (x**3/3 - (x**2/2)*(self.a + self.b) + x*self.a*self.b) + self.C1

    def phi_R(self, x):
        return self.A2 * (x**3/3 - (x**2/2)*(self.a + self.b) + x*self.a*self.b) + self.C2

    def psi_L(self, x):
        return self.A3 * (x - self.b)**2 * (x - self.a)

    def psi_R(self, x):
        return self.A4 * (x - self.a)**2 * (x - self.b)

    # Second Derivatives
    def phi_L_pp(self, x):
        return self.A1 * (2*x - (self.a + self.b))

    def phi_R_pp(self, x):
        return self.A2 * (2*x - (self.a + self.b))

    def psi_L_pp(self, x):
        return self.A3 * (6*x - 4*self.b - 2*self.a)

    def psi_R_pp(self, x):
        return self.A4 * (6*x - 4*self.a - 2*self.b)
```

4.3 Part c and d

```
from scipy.integrate import simpson

# ========================================
# Main FEM Solver
# ========================================
def run_fem(M):
```

```
h = 1.0 / M
nodes = np.linspace(0, 1, M + 1)

# Total DOFs: 2 per node (value and slope).
# Node 0 and Node M are fixed (4 constraints), so internal DOFs = 2*(M-1)
num_total_dofs = 2 * (M + 1)
K_global = np.zeros((num_total_dofs, num_total_dofs))
F_global = np.zeros(num_total_dofs)

for i in range(M):
    a, b = nodes[i], nodes[i+1]
    basis = HermiteBasis(a, b)

    # Quadrature setup for the element
    x_q = np.linspace(a, b, 100)
    funcs = [basis.phi_L, basis.phi_R, basis.psi_L, basis.psi_R]
    derivs = [basis.phi_L_pp, basis.phi_R_pp, basis.psi_L_pp, basis.psi_R_pp]

    # Global DOF mapping: Node i -> (2*i, 2*i+1)
    # Sequence in screenshot 2: c1=phi_L, c2=phi_R, c3=psi_L, c4=psi_R
    map_idx = [2*i, 2*(i+1), 2*i+1, 2*(i+1)+1]

    # Part (c) & (d): Local Stiffness and Load assembly
    for r in range(4):
        # Load vector b using Simpson's
        F_global[map_idx[r]] += simpson(y=f_analytical(x_q) * funcs[r](x_q), x=x_q)

        for c in range(4):
            # Stiffness matrix A
            K_val = simpson(y=derivs[r](x_q) * derivs[c](x_q), x=x_q)
            K_global[map_idx[r], map_idx[c]] += K_val
```

4.4 Part e

```
    # Apply BCs: u(0)=u'(0)=u(1)=u'(1)=0
    # Boundary DOF indices: 0, 1 (node 0) and 2*M, 2*M+1 (node M)
    free_indices = np.arange(2, 2*M)
    K_reduced = K_global[np.ix_(free_indices, free_indices)]
    F_reduced = F_global[free_indices]

    # Part (e): Solve and Reconstruct
    xi_star = solve(K_reduced, F_reduced)
    full_xi = np.zeros(num_total_dofs)
    full_xi[free_indices] = xi_star

    def u_h(x_eval):
        # Determine which element x falls into
        if x_eval >= 1.0: x_eval = 1.0 - 1e-12
        idx = int(x_eval // h)
        a, b = nodes[idx], nodes[idx+1]
        local_basis = HermiteBasis(a, b)
        # coeffs: phi_L, phi_R, psi_L, psi_R
        c = [full_xi[2*idx], full_xi[2*(idx+1)], full_xi[2*idx+1], full_xi[2*(idx+1)+1]]
        return c[0]*local_basis.phi_L(x_eval) + c[1]*local_basis.phi_R(x_eval) + \
               c[2]*local_basis.psi_L(x_eval) + c[3]*local_basis.psi_R(x_eval)

    return np.vectorize(u_h)
```

4.5 Part f

```
# ========================================
# Part (f): Error Analysis and Plotting
# ========================================
M_steps = np.arange(5, 105, 5)
rel_errors = []
x_fine = np.linspace(0, 1, int(1e5))
```

```
u_fine_exact = u_exact(x_fine)
exact_l2 = np.sqrt(np.trapezoid(u_fine_exact**2, x_fine))

print(f"{'M':<5} | {'Relative L2 Error':<20}")
print("-" * 30)

for M in M_steps:
    uh_func = run_fem(M)
    u_approx = uh_func(x_fine)

    l2_error = np.sqrt(np.trapezoid((u_fine_exact - u_approx)**2, x_fine))
    rel_error = l2_error / exact_l2
    rel_errors.append(rel_error)
    print(f"{M:<5} | {rel_error:<20.4e}")

# Plotting
plt.figure(figsize=(8, 5))
plt.loglog(M_steps, rel_errors, 'o-', label='FEM Error')
# Theoretical convergence O(h^4) for L2 norm with cubic Hermite
plt.loglog(M_steps, 0.1 * (M_steps/5)**-4, '--', label='Order 4 Reference')
plt.title("Relative $L^2$ Error vs. Number of Elements")
plt.xlabel("M (Elements)")
plt.ylabel("Relative Error")
plt.grid(True, which="both", ls="-", alpha=0.5)
plt.legend()
plt.savefig("gemini.png", dpi=600)
plt.show()
```

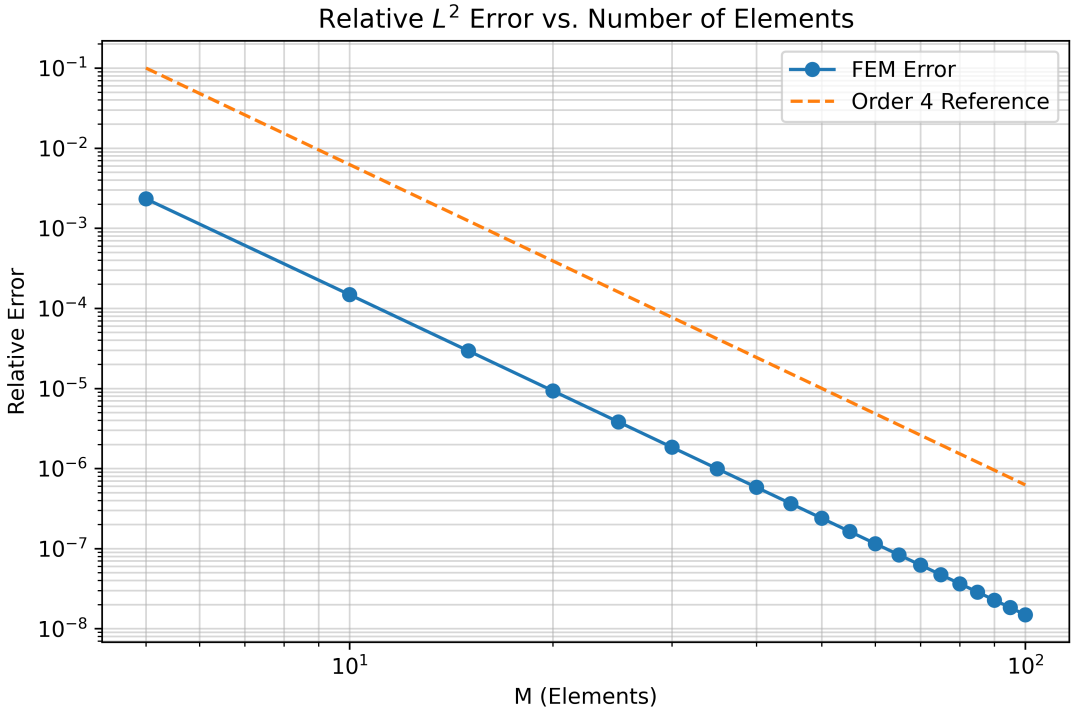

Figure 1: Relative L^2 Error vs. Number of Elements

5 Appendix

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import simpson
from scipy.linalg import solve

# ========================================
# Part (a): Analytical Functions
# ========================================
def u_exact(x):
    return np.sin(np.pi * x)**2

def f_analytical(x):
    return -8 * (np.pi**4) * np.cos(2 * np.pi * x)

# ========================================
# Part (b): Basis Functions from Screenshot 1
# ========================================
class HermiteBasis:
    def __init__(self, a, b):
        self.a, self.b = a, b
        # Constants for phi_L and phi_R
        denom_phi = (a**3 - b**3)/3 - (a+b)/2 * (a**2 - b**2) + a*b*(a-b)
        self.A1 = 1.0 / denom_phi
        self.C1 = -self.A1 * (b**3/3 - (a+b)/2 * b**2 + a*b**2)

        self.A2 = -self.A1 # From symmetry/screenshot 2b
        self.C2 = -self.A2 * (a**3/3 - (a+b)/2 * a**2 + a**2*b)

        # Constants for psi_L and psi_R
        self.A3 = (a - b)**(-2)
        self.A4 = self.A3
```

```
    def phi_L(self, x):
        return self.A1 * (x**3/3 - (x**2/2)*(self.a + self.b) + x*self.a*self.b) + self.C1

    def phi_R(self, x):
        return self.A2 * (x**3/3 - (x**2/2)*(self.a + self.b) + x*self.a*self.b) + self.C2

    def psi_L(self, x):
        return self.A3 * (x - self.b)**2 * (x - self.a)

    def psi_R(self, x):
        return self.A4 * (x - self.a)**2 * (x - self.b)

    # Second Derivatives
    def phi_L_pp(self, x):
        return self.A1 * (2*x - (self.a + self.b))

    def phi_R_pp(self, x):
        return self.A2 * (2*x - (self.a + self.b))

    def psi_L_pp(self, x):
        return self.A3 * (6*x - 4*self.b - 2*self.a)

    def psi_R_pp(self, x):
        return self.A4 * (6*x - 4*self.a - 2*self.b)

# ========================================
# Main FEM Solver
# ========================================
def run_fem(M):
    h = 1.0 / M
    nodes = np.linspace(0, 1, M + 1)
```

```
# Total DOFs: 2 per node (value and slope).
# Node 0 and Node M are fixed (4 constraints), so internal DOFs = 2*(M-1)
num_total_dofs = 2 * (M + 1)
K_global = np.zeros((num_total_dofs, num_total_dofs))
F_global = np.zeros(num_total_dofs)

for i in range(M):
    a, b = nodes[i], nodes[i+1]
    basis = HermiteBasis(a, b)

    # Quadrature setup for the element
    x_q = np.linspace(a, b, 100)
    funcs = [basis.phi_L, basis.phi_R, basis.psi_L, basis.psi_R]
    derivs = [basis.phi_L_pp, basis.phi_R_pp, basis.psi_L_pp, basis.psi_R_pp]

    # Global DOF mapping: Node i -> (2*i, 2*i+1)
    # Sequence in screenshot 2: c1=phi_L, c2=phi_R, c3=psi_L, c4=psi_R
    map_idx = [2*i, 2*(i+1), 2*i+1, 2*(i+1)+1]

    # Part (c) & (d): Local Stiffness and Load assembly
    for r in range(4):
        # Load vector b using Simpson's
        F_global[map_idx[r]] += simpson(y=f_analytical(x_q) * funcs[r](x_q), x=x_q)

        for c in range(4):
            # Stiffness matrix A
            K_val = simpson(y=derivs[r](x_q) * derivs[c](x_q), x=x_q)
            K_global[map_idx[r], map_idx[c]] += K_val

# Apply BCs: u(0)=u'(0)=u(1)=u'(1)=0
# Boundary DOF indices: 0, 1 (node 0) and 2*M, 2*M+1 (node M)
free_indices = np.arange(2, 2*M)
K_reduced = K_global[np.ix_(free_indices, free_indices)]
```

```
    F_reduced = F_global[free_indices]

    # Part (e): Solve and Reconstruct
    xi_star = solve(K_reduced, F_reduced)
    full_xi = np.zeros(num_total_dofs)
    full_xi[free_indices] = xi_star

    def u_h(x_eval):
        # Determine which element x falls into
        if x_eval >= 1.0: x_eval = 1.0 - 1e-12
        idx = int(x_eval // h)
        a, b = nodes[idx], nodes[idx+1]
        local_basis = HermiteBasis(a, b)
        # coeffs: phi_L, phi_R, psi_L, psi_R
        c = [full_xi[2*idx], full_xi[2*(idx+1)], full_xi[2*idx+1], full_xi[2*(idx+1)+1]]
        return c[0]*local_basis.phi_L(x_eval) + c[1]*local_basis.phi_R(x_eval) + \
               c[2]*local_basis.psi_L(x_eval) + c[3]*local_basis.psi_R(x_eval)

    return np.vectorize(u_h)

# ========================================
# Part (f): Error Analysis and Plotting
# ========================================
M_steps = np.arange(5, 105, 5)
rel_errors = []
x_fine = np.linspace(0, 1, int(1e5))
u_fine_exact = u_exact(x_fine)
exact_l2 = np.sqrt(np.trapezoid(u_fine_exact**2, x_fine))

print(f"{'M':<5} | {'Relative L2 Error':<20}")
print("-" * 30)

for M in M_steps:
```

```
    uh_func = run_fem(M)
    u_approx = uh_func(x_fine)

    l2_error = np.sqrt(np.trapezoid((u_fine_exact - u_approx)**2, x_fine))
    rel_error = l2_error / exact_l2
    rel_errors.append(rel_error)
    print(f"{M:<5} | {rel_error:<20.4e}")

# Plotting
plt.figure(figsize=(8, 5))
plt.loglog(M_steps, rel_errors, 'o-', label='FEM Error')
# Theoretical convergence O(h^4) for L2 norm with cubic Hermite
plt.loglog(M_steps, 0.1 * (M_steps/5)**-4, '--', label='Order 4 Reference')
plt.title("Relative $L^2$ Error vs. Number of Elements")
plt.xlabel("M (Elements)")
plt.ylabel("Relative Error")
plt.grid(True, which="both", ls="-", alpha=0.5)
plt.legend()
plt.savefig("gemini.png", dpi=600)
plt.show()
```